Date:-27/10/2026

CO-2-Assessment Tool 3 – ERROR ANALYSIS

Course Code:- DSA0308

Course Name:- Natural Language Processing

Name: - K.Dhanush Reddy **Reg no: -**192324087

QUESTION 1: Morphological Analysis Error in Biomedical Dataset

1. Aim

To identify errors caused by applying the Porter Stemmer to biomedical words and understand the difference between inflectional and derivational morphology.

2. Preprocessing

The biomedical dataset contains words such as infection, infectious, infected, and infect.

The following preprocessing steps are used:

1. Convert text to lowercase.
2. Remove unnecessary punctuation.
3. Tokenize the text into individual words.
4. Apply the Porter Stemmer.
5. Compare the original words with their stems.

3. Python Program

```
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

ps = PorterStemmer()

words = [
   "infection",
   "infectious",
   "infected",
   "infect"
]
for word in words
 print(word, "->", ps.stem(word))
```

4. Analysis of Stemming Errors

Original Word	Expected Meaning or Root	Analysis
Infection	Infect	Noun formed from the root.
Infectious	Infect	Adjective describing the ability to cause infection.
Infected	Infect	Past-tense or participial form.
Infect	Infect	Base verb.

The Porter Stemmer may produce different stems for related words because it removes suffixes according to predefined rules rather than fully understanding their meanings.

For example, *infected* and *infection* may produce the stem infect, while *infectious* may produce a different stem depending on the stemming rules.

5. Difference Between Inflectional and Derivational Morphology

Inflectional morphology changes the grammatical form of a word without creating a new basic word or changing its grammatical category.

Examples:

- Play → played
- Infect → infected
- Book → books

Derivational morphology creates a related word and may change its grammatical category or meaning.

Examples:

- Infect → infection
- Infect → infectious
- Happy → happiness

6. Conclusion

Porter stemming is useful for reducing word variations, but it can remove important information from biomedical terms. Inflectional and derivational morphology should be treated differently because derivational suffixes may contain important information about a word's meaning.

QUESTION 2: Error Analysis of Finite-State Morphological Parser in Social Media Data

1. Aim

To identify errors in a finite-state morphological parser and modify its transitions to recognize words containing multiple prefixes and suffixes.

2. Incorrect Morphological Analysis

S. No.	Word	Correct Morphological Analysis
1	Happiest	Happy + est
2	Unbelievable	Un + believe + able
3	Running	Run + ing
4	Reordering	Re + order + ing
5	Smartphones	Smart + phone + s
6	Unreadable	Un + read + able

3. Why Does the Parser Fail?

It recognizes only one prefix or suffix.

It cannot handle multiple affixes in a single word.

It does not handle spelling changes correctly.

It has a limited vocabulary.

It cannot recognize some irregular word forms.

For example, the word unreadable contains the prefix un- and the suffix -able. A simple parser may fail to identify both affixes.

4. Corrected Python Program

```
def morphological_parser(word):

prefixes = ["un", "re"]
suffixes = ["est", "ing", "able", "s"]

original = word
prefix_found = ""
```

```
suffix_found = ""

for prefix in prefixes:
if word.startswith(prefix) and len(word) > len(prefix):
prefix_found = prefix
word = word[len(prefix):]
break

for suffix in suffixes:
if word.endswith(suffix) and len(word) > len(suffix):
suffix_found = suffix
word = word[:-len(suffix)]
break

return original, prefix_found, word, suffix_found


words = [
"unbelievable",
"running",
"reordering",
"smartphones",
"unreadable"
]

for word in words:
print(morphological_parser(word))
```

Note: This is a simplified parser. Additional rules are required to correctly analyze all six words, including happiest and running.

5. Evaluation Before and After Correction

Parser accuracy is calculated using the following formula:

Accuracy = (Number of Correct Analyses / Total Number of Words) × 100

Evaluation	Correct Analyses	Total Words	Accuracy
Before Correction	3	6	50%
After Correction	6	6	100%

Note: These values are illustrative examples, not actual experimental results from the Sentiment140 dataset.

6. Computational Complexity

For a word of length n, checking a fixed number of prefixes and suffixes takes approximately O(n) time in this simplified implementation. If more transitions and possible analyses are added, the computational requirements may increase.

7. Conclusion

A finite-state morphological parser must recognize multiple prefixes and suffixes and handle spelling changes. Correcting these rules improves morphological analysis and helps process social media text more accurately.

QUESTION 6: ERROR ANALYSIS OF A FINITE-STATE MORPHOLOGICAL PARSER

1. Aim

To find the errors in a simple plural-noun parser and correct it so it can handle regular plurals, -es endings, -ies endings, and common irregular plurals.

2. Errors in the Original Program

The original code uses word[:-2] for every word ending in "s". This removes two characters instead of only the plural ending. It also does not handle words ending in -es or -ies separately, and it treats irregular plurals such as "children" as singular.

Input	Original problem	Expected result
cars	Removes "rs" instead of only "s".	car – Plural Noun
boxes	Does not remove "es" correctly.	box – Plural Noun
cities	Needs the -ies → -y rule.	city – Plural Noun
children	Irregular plural is not recognized.	child – Plural Noun

3. Corrected Python Program

```
irregular_plurals = {
"children": "child",
"people": "person",
```

```
"men": "man",
"women": "woman",
"mice": "mouse",
"teeth": "tooth",
"feet": "foot"
}

def parser(word):
word = word.lower()

if word in irregular_plurals:
return irregular_plurals[word], "Plural Noun"

if word.endswith("ies") and len(word) > 3:
return word[:-3] + "y", "Plural Noun"

if word.endswith(("ches", "shes", "xes", "zes", "sses")):
return word[:-2], "Plural Noun"

if word.endswith("s") and not word.endswith(("ss", "us", "is")):
return word[:-1], "Plural Noun"

return word, "Singular"

words = ["cars", "boxes", "cities", "children"]
for w in words:
print(w, "->", parser(w))
```

This program uses simple rules and a small dictionary of common irregular plurals. It is an educational example, not a complete English morphological parser.

4.Output:-

```
cars -> ('car', 'Plural Noun')
boxes -> ('box', 'Plural Noun')
cities -> ('city', 'Plural Noun')
children -> ('child', 'Plural Noun')
```

These are expected outputs from the corrected rules for the four words provided in the question. The code is not connected to WordNet or another external lexicon.

5. Limitations of Rule-Based Parsing

1. English has irregular forms that cannot be handled by one suffix rule.
2. Some words ending in “s” are singular, such as “news”, so rules may make

mistakes.
3. A dictionary can be incomplete and may not recognize new words.
4. Spelling and meaning can depend on context.

6. Conclusion

Correct plural rules and an irregular-word dictionary help the parser identify common plural nouns more accurately, but a larger lexicon and context are needed for reliable English analysis.

QUESTION 7: ERROR ANALYSIS OF MORPHOLOGICAL FEATURE EXTRACTION

1. Aim

To identify why stemming after feature extraction is incorrect and correct the text-processing pipeline so that words are stemmed before the vocabulary is created.

2. Error in the Original Program

The original program first runs CountVectorizer on the unstemmed documents and creates the vocabulary. It then stems only the feature names. The matrix X still contains columns for the original words, so the feature names no longer match the columns correctly. Similar words can also remain as separate columns.

3. Corrected Python Program

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
"running runners runs",
"studies studied studying",
"organization organized organizer"
]

def stem_text(text):
return " ".join(stemmer.stem(word) for word in text.lower().split())

# Stem the documents BEFORE creating the vocabulary
stemmed_documents = [stem_text(doc) for doc in documents]

vectorizer = CountVectorizer()
```

```
X = vectorizer.fit_transform(stemmed_documents)

print("Stemmed documents:")
for doc in stemmed_documents:
print(doc)

print("Vocabulary after stemming:")
print(vectorizer.get_feature_names_out())
print("Vocabulary size after stemming:", len(vectorizer.get_feature_names_out()))
```

For more robust real-world text, use a tokenizer that handles punctuation and contractions instead of split(). The key correction is to stem the documents before calling fit_transform().

4. Comparison

Measure	Original pipeline	Corrected pipeline
Order of processing	Create vocabulary, then stem feature names.	Stem document text, then create vocabulary.
Vocabulary	Original word forms remain separate in X.	Word forms with the same Porter stem can share a feature.
Feature consistency	Feature names may not match the columns in X.	Feature names and matrix columns stay aligned.
Vocabulary size	Usually larger for related word forms.	May be smaller after related forms are merged.
Classification accuracy / processing time	Not measured in the supplied short example.	Must be measured on the same real-world dataset and model for a fair comparison.

The supplied three-document example can demonstrate the corrected pipeline, but it does not provide a real-world classification accuracy result. To compare accuracy and processing time, both pipelines must be tested with the same train/test split, classifier, and dataset such as 20 Newsgroups.

5. Why the Correction Helps

Stemming before vectorization combines many related word forms into shared features. This can reduce vocabulary size and sparsity and may improve learning efficiency. However, stemming can also remove useful meaning, so the effect on accuracy should be measured rather than assumed.

6. Conclusion

Applying stemming before CountVectorizer keeps the feature matrix and vocabulary consistent and can reduce redundant features, making text classification and information retrieval more efficient.